24.2 CDP 层实现

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~170k tokens，输出 ~5k tokens（本节）

上一节我们了解了浏览器控制的整体架构。这一节深入 CDP（Chrome DevTools Protocol）层——OpenClaw 如何通过 WebSocket 与 Chrome 通信，执行低级浏览器操作。

24.2.1 CDP 连接管理

WebSocket URL 规范化

Chrome 的 CDP 接口通过 `/json/version` 端点暴露 WebSocket URL。但这个 URL 可能需要规范化处理：

```typescript
// src/browser/cdp.ts — normalizeCdpWsUrl
export function normalizeCdpWsUrl(wsUrl: string, cdpUrl: string): string {
  const ws = new URL(wsUrl);
  const cdp = new URL(cdpUrl);

  // 如果 ws 是 loopback 但 cdp 不是 → 替换主机名
  //  (远程 Chrome 可能返回 127.0.0.1 作为 ws 地址)
  if (isLoopbackHost(ws.hostname) && !isLoopbackHost(cdp.hostname)) {
    ws.hostname = cdp.hostname;
    ws.port = cdp.port;
    ws.protocol = cdp.protocol === "https:" ? "wss:" : "ws:";
  }

  // 协议升级：如果 cdp 是 https，ws 也应该是 wss
  if (cdp.protocol === "https:" && ws.protocol === "ws:") {
    ws.protocol = "wss:";
  }

  // 传递认证信息
  if (!ws.username && !ws.password && (cdp.username || cdp.password)) {
    ws.username = cdp.username;
    ws.password = cdp.password;
  }

  // 传递查询参数
  for (const [key, value] of cdp.searchParams.entries()) {
    if (!ws.searchParams.has(key)) {
      ws.searchParams.append(key, value);
    }
  }
  return ws.toString();
}
```

这个函数处理了远程 CDP 连接的常见问题：Chrome 返回的 `webSocketDebuggerUrl` 通常是 `ws://127.0.0.1:9222/...`（因为 Chrome 只知道自己在本地），但如果 OpenClaw 通过反向代理或 SSH 隧道访问远程 Chrome，就需要将 URL 替换为实际的远程地址。

CDP Socket 封装

`withCdpSocket()` 是 CDP 通信的核心原语：

```typescript
// src/browser/cdp.helpers.ts — withCdpSocket（简化）
export async function withCdpSocket<T>(
  wsUrl: string,
  fn: (send: CdpSendFn) => Promise<T>,
): Promise<T> {
  const ws = new WebSocket(wsUrl, { handshakeTimeout: 5000 });

  // 等待连接建立
  await new Promise((resolve, reject) => {
    ws.once("open", resolve);
    ws.once("error", reject);
  });

  // 创建 CDP 命令发送器
  const { send, closeWithError } = createCdpSender(ws);

  try {
    return await fn(send);
  } finally {
    ws.close();
  }
}
```

`createCdpSender()` 实现了 CDP 的 JSON-RPC 协议：

```typescript
// src/browser/cdp.helpers.ts — createCdpSender
function createCdpSender(ws: WebSocket) {
  let nextId = 1;
  const pending = new Map<number, Pending>();

  const send: CdpSendFn = (method, params?) => {
    const id = nextId++;
    ws.send(JSON.stringify({ id, method, params }));
    return new Promise((resolve, reject) => {
      pending.set(id, { resolve, reject });
    });
  };

  ws.on("message", (data) => {
    const parsed = JSON.parse(data) as CdpResponse;
    const p = pending.get(parsed.id);
    if (!p) return;
    pending.delete(parsed.id);
    if (parsed.error?.message) {
      p.reject(new Error(parsed.error.message));
    } else {
      p.resolve(parsed.result);
    }
  });

  return { send, closeWithError };
}
```

> **衍生解释**：CDP 使用 JSON-RPC over WebSocket 协议。每个请求有一个递增的 `id`，服务端回复时带上相同的 `id`，客户端通过 `id` 将请求和响应配对。这是一个经典的**请求-响应多路复用**模式——一个 WebSocket 连接可以同时承载多个并发的请求/响应对。`pending` Map 存储了每个未完成请求的 Promise 回调。

24.2.2 CDP 辅助函数

截图

```
// src/browser/cdp.ts — captureScreenshot
export async function captureScreenshot(opts: {
  wsUrl: string;
  fullPage?: boolean;
  format?: "png" | "jpeg";
  quality?: number;
}): Promise<Buffer> {
  return await withCdpSocket(opts.wsUrl, async (send) => {
    await send("Page.enable");

    let clip;
    if (opts.fullPage) {
      // 获取页面完整尺寸
      const metrics = await send("Page.getLayoutMetrics");
      const size = metrics?.cssContentSize ?? metrics?.contentSize;
      if (size.width > 0 && size.height > 0) {
        clip = { x: 0, y: 0, width: size.width, height: size.height, scale: 1 };
      }
    }

    const result = await send("Page.captureScreenshot", {
      format: opts.format ?? "png",
      quality: opts.format === "jpeg" ? opts.quality ?? 85 : undefined,
      fromSurface: true,
      captureBeyondViewport: true,
      ...(clip ? { clip } : {}),
    });

    return Buffer.from(result.data, "base64");
  });
}
```

全页截图的关键：先通过 `Page.getLayoutMetrics` 获取页面的完整内容尺寸（可能远大于视口），然后将该尺寸作为 `clip` 参数传入截图命令。 `captureBeyondViewport: true` 确保能截取超出当前视口的内容。

JavaScript 执行

```
// src/browser/cdp.ts — evaluateJavaScript
export async function evaluateJavaScript(opts: {
  wsUrl: string;
  expression: string;
  awaitPromise?: boolean;
  returnByValue?: boolean;
}): Promise<{ result: CdpRemoteObject; exceptionDetails?: CdpExceptionDetails }> {
  return await withCdpSocket(opts.wsUrl, async (send) => {
    await send("Runtime.enable").catch(() => {});
    const evaluated = await send("Runtime.evaluate", {
      expression: opts.expression,
      awaitPromise: Boolean(opts.awaitPromise),
      returnByValue: opts.returnByValue ?? true,
      userGesture: true,           // 模拟用户操作上下文
      includeCommandLineAPI: true, // 启用 $、$$、copy() 等调试辅助函数
    });
    return { result: evaluated.result, exceptionDetails: evaluated.exceptionDetails };
  });
}
```

`userGesture: true` 很重要——某些浏览器 API（如 `navigator.clipboard.writeText()` 、 `Notification.requestPermission()` ）要求在"用户手势"上下文中调用。通过此参数，CDP 模拟了用户交互环境。

标签页创建

```
// src/browser/cdp.ts — createTargetViaCdp
export async function createTargetViaCdp(opts: {
  cdpUrl: string;
  url: string;
}): Promise<{ targetId: string }> {
  // 获取浏览器级别的 WebSocket URL（非标签页级别）
  const version = await fetchJson(appendCdpPath(opts.cdpUrl, "/json/version"));
  const wsUrl = normalizeCdpWsUrl(version.webSocketDebuggerUrl, opts.cdpUrl);

  return await withCdpSocket(wsUrl, async (send) => {
    const created = await send("Target.createTarget", { url: opts.url });
    return { targetId: created.targetId };
  });
}
```

注意这里使用的是**浏览器级别**的 WebSocket（通过 `/json/version` 获取），而非标签页级别的 WebSocket。`Target.createTarget` 是一个浏览器级别的 CDP 命令。

24.2.3 Target ID 管理与标签页操作

什么是 Target ID

在 CDP 中，每个浏览器标签页（以及 Service Worker、iframe 等）都是一个 **Target**，由一个唯一的 `targetId` 标识。OpenClaw 使用 `targetId` 来定位和操作具体的标签页。

模糊匹配

```
// src/browser/target-id.ts — resolveTargetIdFromTabs
export function resolveTargetIdFromTabs(
  input: string,
  tabs: Array<{ targetId: string }>,
): TargetIdResolution {
  const needle = input.trim();

  // 1. 精确匹配
  const exact = tabs.find(t => t.targetId === needle);
  if (exact) return { ok: true, targetId: exact.targetId };

  // 2. 前缀匹配（大小写不敏感）
  const matches = tabs
    .map(t => t.targetId)
    .filter(id => id.toLowerCase().startsWith(needle.toLowerCase()));

  if (matches.length === 1) return { ok: true, targetId: matches[0] };
  if (matches.length === 0) return { ok: false, reason: "not_found" };
  return { ok: false, reason: "ambiguous", matches };
}
```

返回类型是一个判别联合（Discriminated Union）：

```
type TargetIdResolution =
  | { ok: true; targetId: string }
  | { ok: false; reason: "not_found" | "ambiguous"; matches?: string[] };
```

> **衍生解释**：Target ID 是 Chrome 内部生成的 UUID 格式字符串（如 `8D3E4F7A-2B1C-4D5E-9F0A-1B2C3D4E5F6A`）。在 Agent 与用户的对话中，传递完整的 UUID 不太方便。前缀匹配允许 Agent 只用前几个字符就能唯一定位一个标签页——类似 Git 的短 commit hash。`ambiguous` 结果意味着前缀匹配到多个标签页，需要更长的前缀。

标签页列表

标签页信息通过 CDP 的 `/json/list` 端点获取：

```
// 服务器上下文中的 listTabs（简化）
const listTabs = async (): Promise<BrowserTab[]> => {
  if (!profile.cdpIsLoopback) {
    // 远程浏览器：通过 Playwright 列出页面
    const pages = await listPagesViaPlaywright({ cdpUrl: profile.cdpUrl });
    return pages.map(p => ({
      targetId: p.targetId,
      title: p.title,
      url: p.url,
      type: "page",
    }));
  }
  // 本地浏览器：直接调用 CDP HTTP 接口
  const tabs = await fetchJson(appendCdpPath(profile.cdpUrl, "/json/list"));
  return tabs.filter(t => t.type === "page").map(t => ({
    targetId: t.id,
    title: t.title,
    url: t.url,
    wsUrl: normalizeWsUrl(t.webSocketDebuggerUrl, profile.cdpUrl),
  }));
};
```

本地 vs 远程的处理差异：

- **本地**：直接 HTTP 调用 `/json/list`，效率更高
- **远程**：通过 Playwright 的持久连接列出页面，避免每次都建立新的短暂连接

24.2.4 认证与安全

CDP 通信中的认证通过 `getHeadersWithAuth()` 统一处理：

```
// src/browser/cdp.helpers.ts — 认证头注入
export function getHeadersWithAuth(url: string, headers = {}) {
  // 1. 检查 Chrome Extension Relay 认证
  const relayHeaders = getChromeExtensionRelayAuthHeaders(url);
  const merged = { ...relayHeaders, ...headers };

  // 2. 检查 URL 中的 Basic Auth
  const parsed = new URL(url);
  if (parsed.username || parsed.password) {
    const auth = Buffer.from(`${parsed.username}:${parsed.password}`)
      .toString("base64");
    return { ...merged, Authorization: `Basic ${auth}` };
  }

  return merged;
}
```

支持三种认证方式：

1. **Chrome Extension Relay Token**——通过自定义 header `x-openclaw-relay-token` 传递
2. **URL Basic Auth**—— `http://user:pass@host:port/` 格式
3. **无认证**——本地 loopback 连接通常不需要

本节小结

1. **CDP 通信基于 JSON-RPC over WebSocket**——每个请求/响应通过递增 `id` 配对，支持并发多路复用
2. **WebSocket URL 规范化**处理远程连接的地址替换、协议升级、认证传递
3. **`withCdpSocket()` 是一次性连接模式**——打开 WebSocket、执行操作、关闭，适合截图、JS 执行等短期操作
4. **截图支持全页模式**——通过 `Page.getLayoutMetrics` 获取完整内容尺寸，`captureBeyondViewport` 截取视口外内容
5. **JS 执行使用 `userGesture: true`** ——模拟用户交互上下文，解锁受限的浏览器 API
6. **Target ID 支持前缀模糊匹配**——Agent 不需要记忆完整的 UUID，类似 Git 短 hash
7. **三种认证方式**：Extension Relay Token、URL Basic Auth、无认证